Exercise 4: An MCP Server and a Subagent

Session: 4 — MCP Servers & External Tools
Duration: 30 minutes, plus a 5-minute closing round
Project: The same BookStore API.

Goal

In this exercise you:

1. Connect an MCP server that lets Claude read the bookstore database. You ask one database question before and after, and compare the answers.
2. Write an agent file for a subagent that checks the code for security problems. You write two lines: when Claude uses it, and which tools it may use.

Start here

- Open a terminal in the `bookstore` folder. Stay in this folder for the whole exercise.
- Training mode is on. Some prompts start with a tag in square brackets. The tag switches training mode off for that one prompt. Copy those prompts exactly, tag included. Never edit `CLAUDE.md`.
- Write your notes on paper or in a text file, not in the chat.
- Stuck? Type *just tell me*. The trainer tells you when to start task 4.

Tasks

1. Ask without the MCP server

This task produces a `permissions.deny` rule in `.claude/settings.json`, a prediction and three notes. This is round one.

1. **Add a deny rule**. Start `claude` and paste:

```
[Exercise 4 experiment — execute directly, no leading questions.] Add the rule
Bash(sqlite3 *) to the deny list under permissions in .claude/settings.json. Keep
everything else in that file. Create the file if it does not exist.
```

   Keep the hook from session 3 in the file.

2. **Write a prediction**. In the next step you ask three things: the number of books, the author with the most books, and a SQL query. Write one sentence. Which of the three can Claude get right from the source code alone, and which will be a guess?

3. **Ask**. Paste:

```
[Exercise 4 experiment — execute directly, no leading questions.] How many books are in the bookstore database? Which author has the most books? Write a SQL query that returns all books with their author name and average rating, sorted by rating, highest first.
```

   Write down three notes: the book count, the author, and how sure Claude says it is (or "did not say"). Then check one more thing. Does Claude say that its answers come from the code, not from the database?

Why a deny rule. `CLAUDE.md` asks Claude not to open `store.db`, but Claude can still decide to do it. A `permissions.deny` rule makes Claude Code block the command before it runs (*MCP Permissions: Allow, Ask, Deny*).

Done when: your prediction and three notes are written, and Claude did not open `store.db`.

2. Register the MCP server and approve it

This task produces `.mcp.json` and a connected server.

The server has two tools: `get_table_definitions` and `execute_query`. It only runs `SELECT` queries. It is one Python file, `mcp-sqlite/server.py`. It reads `store.db`, which the app creates the first time it runs.

1. **Leave Claude**. Type `/exit`. The next commands go to the terminal, not to Claude. No `store.db` file in the folder? Run `./gradlew run`, wait for `listening on :8080`, and stop it with Ctrl+C.

2. **Register the server**. Run this in the terminal, on one line:

```
claude mcp add --scope project sqlite-bookstore -- uv run mcp-sqlite/server.py store.db
```

   The part after `--` is the command that starts the server. It uses `uv`, which you installed in the preparation. With `--scope project`, Claude Code writes the command to `.mcp.json` in this folder (*Configuring an MCP Server*). Open

`.mcp.json` and find the command you typed. Git ignores `.mcp.json` in this course, so the file stays on your machine.

`uv` missing or failing? Run `claude mcp remove sqlite-bookstore`. Then run the command above again, with `python3` (on Windows: `python`) in place of `uv run`. The server needs only Python, no packages.

3. **Restart and approve**. Start `claude`. Claude Code asks whether it may use the server from `.mcp.json`. Answer yes. Did you answer no? Type `/exit`, run `claude mcp reset-project-choices` in the terminal, and start `claude` again.

4. **Check**. Type `/mcp`. `sqlite-bookstore` shows as connected, with both tools. Not connected? Type `/exit` and run `claude mcp list` in the terminal. It shows the error. The usual causes: `store.db` is missing (see step 1), or the command in `.mcp.json` has a typing error ("*Claude Cannot See the Database*").

Why the approval. A project file can start any program on your machine. So Claude Code asks you once per project, and saves your answer in `.claude/settings.local.json`.

Done when: `/mcp` shows `sqlite-bookstore` connected, with two tools.

3. Ask again, with the MCP server

This task is round two. It produces three more notes.

1. **Same question**. Paste the prompt from task 1, step 3 again, tag included. Watch the tool calls: `get_table_definitions` first, then `execute_query`. The names appear as `mcp__sqlite-bookstore__execute_query` ("*Claude Cannot See the Database*").

2. **Compare**. Write down the same three notes. Which answers changed? Was your prediction right?

3. **Ask one thing the code cannot tell**. Paste this prompt and note the title Claude finds:

```
[Exercise 4 experiment — execute directly, no leading questions.] Which book has the highest average rating? Show its title, its author and the text of its three best reviews.
```

Why the two rounds differ. Without the server, Claude reads the code that fills the database at first start. The column names come from that code, so they can be right. The numbers are a guess. With the server, Claude makes two read-only tool calls,

and the answer comes from the real data.

Done when: you have three notes per round, and you can say which answer in round one was a guess.

4. Create the security-auditor subagent

This task produces the agent file `.claude/agents/security-auditor.md` and the report `docs/security-audit.md`. A starter file holds the instructions for the check, so you write only the two frontmatter lines in step 2.

The frontmatter is the first block of the agent file, between two `---` lines (*An Agent File Encodes a Task*).

1. **Let Claude create the file**. Paste:

```
[Exercise 4 experiment — execute directly, no leading questions.] Copy
exercises/starters/security-auditor.md to .claude/agents/security-auditor.md. Change
nothing in the content.
```

2. **Write the two lines**. Open `.claude/agents/security-auditor.md` in your editor. Replace the two `(I write this)` values.
   - `description`: Claude reads only this line when it decides whether to start the agent (*How Agents Get Invoked*). Say what the agent checks. Say when Claude should use it: what a user might ask for, and which code change should start it. A weak version is `Reviews code for security issues.` Write a better one, in your own words. The rule from the slide *Which Subagent Description Invokes Reliably?* applies: write the trigger, not just the job.
   - `tools`: tool names, separated by commas, as in the agent file on the slide *An Agent File Encodes a Task*. The agent must never change code. Give it the smallest set that still lets it find and read files.

   Optional, if you have time: run `/mcp-coach 4` (4 is the task number). It reads your description, asks one question about it, and does not write the line for you.

3. **Test the trigger**. Type `/exit` and start `claude` again. Do not use `/clear`: agents load only at start. Paste this prompt. It does not name the agent:

```
[Exercise 4 experiment — execute directly, no leading questions.] Do a security audit of
the bookstore API. Save the full report, exactly as it was produced, to docs/security-
audit.md.
```

Watch the tool calls. When the agent starts, Claude prints a line with the name `security-auditor`, and the check runs inside that agent. Then your description works.

Do you only see Claude's own `Read` and `Grep` calls? Then Claude did the check itself, and your description did not start the agent. Improve the description, type `/exit`, start `claude`, and ask again. After two tries without the agent, add "*use the security-auditor agent*" to the prompt. Then you still get a report.

4. **Check the report**. Open `docs/security-audit.md`. Near the top, find the line `**Audited by**: security-auditor`.

The starter file asks the agent to write that line. The agent cannot write files, so your own Claude session saved the report.

Done when: both `(I write this)` values are replaced, and `docs/security-audit.md` has the `Audited by` line.

Bonus (only if time remains)

Try to write through the server. Paste:

```
[Exercise 4 experiment — execute directly, no leading questions.] Set the rating of review 1 to 5 in the database.
```

`execute_query` runs `SELECT` only, and the server opens the database read-only. Claude should tell you that it cannot do this. The server can read and nothing more. That is least privilege: the server gets only the rights it needs.

Read the server. Open `mcp-sqlite/server.py`. You do not need to know Python. Find the two tool names, and find `mode=ro`, where the database is opened read-only. The whole server is one file of about 200 lines. Compare it with the example on the slide *Building a Custom MCP Server*.

Run the plugin. The folder `bookstore-plugin`, inside `bookstore`, holds the skills from exercise 3, the hook, a finished security-auditor agent and the SQLite server, packaged as one plugin (*Plugins: Bundling It All Together*).

1. Type `/exit`.
2. Start Claude with `claude --plugin-dir bookstore-plugin`.
3. Type `/help`. The skills now appear as `/bookstore-plugin:commit` and `/bookstore-plugin:changelog`, next to your own.

4. Type `/mcp`. The plugin's server is connected. Claude Code did not ask for approval.

The `README.md` in `bookstore-plugin` explains what is different from your files.

Closing round

First run `/verify-exercise 4` in Claude. It reads `.mcp.json`, calls the server, reads your agent file and the report, and asks you one question. Read its report while the trainer asks the room.

Have these answers ready:

- Which answer in round one was a guess, and how did round two show it?
- Did Claude call `get_table_definitions` before every query, or only once?
- Did the subagent start by itself? Which words in your `description` did that?
- One thing you would tell someone who skipped this session.